Set 3:

1. **On-site Customer:** Client representative always present on-site to clear your questions
   Adv: No guesswork / Customer satisfaction
   Disadv: Client not free / Client is not competent / Client makes a lot of changes

   Metaphor: Explain the project using a simple story or description to make the entire team understand the idea of the project
   Adv: Each team member gets an idea of the project
   Disadv: Not much informative

2. **Code Refactoring:** Fix internal code without changing external functionality, removing bad smell of code

   Agile Philosophy: Highest priority is customer satisfaction, give quick updates to client to get their feedback
   Alignment: Sometimes we cannot properly code while giving quick updates, so we can use refactoring to fix that code later

   Simple Design: Keep the design simple, build only what is needed
   Alignment: We cannot always afford simple design, we will need to improve the code, so we can later refactor and improve the simple design

   Coding Standard: Maintain basic coding standards and conventions while coding
   Alignment: If we maintain coding standards, then the true purpose of refactoring can be achieved - that is focus on fixing bad smells. If we code poorly, then refactoring will be needed to fix the coding standards

Set 4:

1. **Collective Code Ownership:** Code must be accessible and editable to all team members, no one can own the code or any modules.
   Adv: Can work around errors of other developers / Faster coding
   Disadv: Weak developers can ruin a code while editing

   Continuous Integration: Regularly(once or twice a day) update individual codes into the main code base
   Adv: Less errors, regular merging creates small errors which can be easily fixed
   Disadv: Not always possible to integrate regularly

2. **Problem 1:** Cancellation of Project
   Solution: On-site customer (regular monitoring from client, knows about the project properly, less chance of rejection), Small releases (client satisfaction, less chance of mistakes and rejection)

   Problem 2: Errors in Project
   Solution: Test Driven Development (Regular testing, less chance of error), Code Refactoring (Less errors as code is fixed), Pair Programming (Two minds create better quality code)

   Problem 3: Staff Turnover
   Solution: Daily Stand-up Meeting (New employee can learn from the meetings daily), Pair Programming (Can pair new employee with experience employee, so that they learn about the project), On-Site Customer (Can learn about the project from the client representative)